

测试计划与测试用例说明——预制菜WMS仓储管理系统

1. 测试概述

1.1 测试目标

- 验证所有核心业务流程（入库、出库、质检、盘点、调整、次品处理）的功能正确性
- 验证32个控制器142个API接口的响应正确性
- 验证前端8个页面+微信小程序18个页面的UI交互正确性
- 验证数据库22张表的完整性和事务一致性
- 验证JWT认证、权限控制、AES加密的安全性

1.2 测试范围

子系统	测试内容	优先级
后端API	142个接口的功能测试	P0
核心业务流程	入库、出库、质检、盘点、调整、次品处理	P0
前端Web端	8个页面的UI交互	P1
微信小程序	18个页面的功能验证	P1
认证安全	JWT登录、权限校验、密码加密	P0
数据看板	4个图表的API数据准确性	P2
AI助手	DeepSeek对话与数据注入	P2
跨平台	Web端+小程序数据同步	P2

1.3 测试环境

环境	配置
后端服务器	Spring Boot 3, Java 17, localhost:8088
数据库	MySQL 8.0, Redis 6, localhost
Web前端	Vite Dev Server, Chrome/Edge浏览器, 1920×1080
微信小程序	微信开发者工具 Stable 1.06+, SDK 3.16.2
API测试工具	Postman / Swagger UI

2. 测试策略

2.1 测试方法

测试类型	方法	覆盖范围
单元测试	JUnit 5 + SpringBootTest (PmfApplicationTests)	Spring上下文加载验证
接口测试	Swagger UI + Postman手动测试	142个API端点
功能测试	手工黑盒测试 (按测试用例执行)	6大核心业务流程
UI测试	手工页面验证 + 小程序真机调试	Web端8页面 + 小程序18页面
安全测试	手工验证JWT/权限/加密	认证+授权+传输加密

2.2 测试优先级定义

级别	定义	说明
P0	核心功能	必须通过，否则系统不可用
P1	重要功能	应该通过，影响主要用户体验
P2	一般功能	建议通过，影响辅助功能

3. 测试用例

3.1 认证模块测试用例

TC-AUTH-001: 正确用户名密码登录

项	内容
测试编号	TC-AUTH-001
测试模块	认证登录
优先级	P0
前置条件	数据库存在用户admin/123456
测试步骤	1. 打开登录页 2. 输入用户名admin 3. 输入密码123456(自动AES加密) 4. 点击登录
预期结果	返回code=200, data包含用户信息, token有效, 前端跳转到主页, localStorage存储token和userInfo
接口	POST /auth/login

TC-AUTH-002: 错误密码登录

项	内容
测试编号	TC-AUTH-002
测试模块	认证登录
优先级	P0
前置条件	数据库存在用户admin
测试步骤	使用错误密码登录
预期结果	返回code=1001, msg="用户名或密码错误", 前端显示错误提示

TC-AUTH-003: JWT令牌验证

项	内容
测试编号	TC-AUTH-003
测试模块	JWT认证
优先级	P0
测试步骤	1. 不带token访问/user/personnel 2. 带过期token访问
预期结果	返回code=1002, msg="令牌已过期或者失效"

TC-AUTH-004: 密码传输加密

项	内容
测试编号	TC-AUTH-004
测试模块	AES加密
优先级	P0
测试步骤	1. 登录时抓包查看请求体 2. 检查password字段
预期结果	password为Base64编码的密文(非明文), 后端能正确解密

TC-AUTH-005: 密码存储加密

项	内容
测试编号	TC-AUTH-005
测试模块	BCrypt存储
优先级	P0
测试步骤	1. 注册新用户 2. 查看数据库中password字段
预期结果	数据库中存储的是BCrypt密文(2a10\$开头), 不是明文

3.2 入库业务测试用例

TC-INBOUND-001: 创建入库任务

项	内容
测试编号	TC-INBOUND-001
测试模块	入库业务
优先级	P0
前置条件	存在可用批次(状态=待入库)和仓库员工
测试步骤	1. 进入任务中心→操作中心 2. 点击"入库" 3. 选择入库类型(采购入库) 4. 选择可用批次 5. 选择执行人 6. 设置优先级+截止日期 7. 提交
预期结果	生成入库单号(I+日期+流水), 创建work_task记录, 创建operation_log, 入库单状态=已审核
接口	POST /inbound/create

TC-INBOUND-002: 执行入库(完成)

项	内容
测试编号	TC-INBOUND-002
测试模块	入库业务
优先级	P0
前置条件	存在已审核的入库任务
测试步骤	1. 仓库员工进入待办列表 2. 点击入库任务的"去完成" 3. 选择仓库 4. 从可用库位中勾选所需数量库位 5. 确认执行
预期结果	库位标记占用, inventory记录创建(状态=正常), 批次→正常, 货物数量增加, zone/warehouse可用数减少, operation_log记录
涉及接口	PUT /inbound/complete/{taskId}

TC-INBOUND-003: 库位分配算法验证

项	内容
测试编号	TC-INBOUND-003
测试模块	入库业务
优先级	P0
前置条件	批次初始数量=100
测试步骤	1. 执行入库 2. 查看需要选择的库位数
预期结果	需要选择 $\text{ceil}(100/10) = 10$ 个库位, 每个库位分配10单位

TC-INBOUND-004: 储存条件匹配

项	内容
测试编号	TC-INBOUND-004
测试模块	入库业务
优先级	P1
前置条件	货物储存条件=冷冻, 系统有冷冻库和一般仓库
测试步骤	1. 创建冷冻货物入库 2. 执行入库查看可选库位
预期结果	只显示冷冻库的库位, 不显示一般仓库的库位

TC-INBOUND-005: 事务回滚验证

项	内容
测试编号	TC-INBOUND-005
测试模块	入库业务
优先级	P1
测试步骤	1. 模拟执行入库时某步骤失败(如库位被锁定) 2. 检查数据库
预期结果	所有相关表数据回滚到操作前状态(@Transactional保护)

3.3 出库业务测试用例

TC-OUTBOUND-001: 创建出库任务

项	内容
测试编号	TC-OUTBOUND-001
测试模块	出库业务
优先级	P0
前置条件	存在正常库存(inventory.status=正常, quantity>0)
测试步骤	1. 操作中心→出库 2. 选择货物(FIFO排序) 3. 选择批次 4. 选择来源库位 5. 输入出库数量 6. 提交
预期结果	生成出库单号, 源库位锁定(lock_status=锁定, lock_purpose=出库), inventory.locked_quantity增加

项	内容
接口	POST /outbound/create

TC-OUTBOUND-002: FIFO出库验证

项	内容
测试编号	TC-OUTBOUND-002
测试模块	出库业务
优先级	P1
前置条件	同货物有多个批次, 不同有效期
测试步骤	1. 创建出库选择货物 2. 查看库存列表排序
预期结果	库存列表按expiry_date升序排列(最早过期的在前面)

TC-OUTBOUND-003: 执行出库(完成)

项	内容
测试编号	TC-OUTBOUND-003
测试模块	出库业务
优先级	P0
前置条件	存在已审核的出库任务
测试步骤	1. 执行人进入待办 2. 确认出库
预期结果	inventory.quantity减少, inventory.locked_quantity减少, goods.quantity减少, batch.remaining_quantity减少, 库位解锁

项	内容
涉及接口	PUT /outbound/complete/{taskId}

TC-OUTBOUND-004: 库存归零自动清理

项	内容
测试编号	TC-OUTBOUND-004
测试模块	出库业务
优先级	P1
前置条件	某库位库存=出库数量
测试步骤	1. 执行出库使该库位库存归零 2. 检查库位状态和批次
预期结果	location.is_occupied=0, zone/warehouse.available_slots增加, 若batch.remaining归零则级联删除

3.4 质检业务测试用例

TC-QC-001: 创建来料检任务

项	内容
测试编号	TC-QC-001
测试模块	质检业务
优先级	P0
前置条件	存在待入库的库存
测试步骤	1. 操作中心→质检 2. 选来料检 3. 选批次(仅显示待入库) 4. 选质检员 5. 提交
预期结果	生成质检单号(C+日期+流水), 类型=来料检
接口	POST /qualityCheck/create

TC-QC-002: 质检合格

项	内容
测试编号	TC-QC-002
测试模块	质检业务
优先级	P0
测试步骤	1. 质检员打开质检任务 2. 选择"合格" 3. 提交
预期结果	batch.status='正常', inventory.status保持'正常'

TC-QC-003: 质检不合格

项	内容
测试编号	TC-QC-003
测试模块	质检业务
优先级	P0
测试步骤	1. 质检员选择"不合格" 2. 填写缺陷原因 3. 提交
预期结果	batch.status='报废', inventory.status='待报废'

TC-QC-004: 不合格必须填写原因

项	内容
测试编号	TC-QC-004
测试模块	质检业务
优先级	P1
测试步骤	选择"不合格"但不填原因, 点击提交
预期结果	前端拦截, 提示"不合格请填写原因"

3.5 盘点业务测试用例

TC-CHECK-001: 按库位盘点

项	内容
测试编号	TC-CHECK-001
测试模块	盘点业务
优先级	P0
测试步骤	1. 操作中心→库存盘点 2. 选"按库位盘点" 3. 选库位 4. 提交
预期结果	生成盘点单号(P+日期+流水), 明细包含该库位所有库存记录(含账面数量)

接口	POST /inventoryCheck/create
----	-----------------------------

TC-CHECK-002: 盘点差异处理

项	内容
测试编号	TC-CHECK-002
测试模块	盘点业务
优先级	P0
前置条件	库位库存账面=20
测试步骤	1. 录入实盘数=15 2. 完成盘点
预期结果	detail_status='盘亏', difference=-5, inventory.quantity更新为15, goods.quantity重算

TC-CHECK-003: 盘点无差异

项	内容
测试编号	TC-CHECK-003
测试模块	盘点业务
优先级	P0
测试步骤	实盘数=账面数
预期结果	detail_status='正常', difference=0

3.6 Web端UI测试用例

TC-UI-001: 登录页UI

测试编号	TC-UI-001	优先级	P1
	测试步骤	1. 打开 http://localhost:8080 2. 检查页面布局 3. 输入空用户名密码点击登录	
	预期结果	左侧品牌图片, 右侧登录表单; 空输入时按钮禁用	

TC-UI-002: 侧边栏导航

测试编号	TC-UI-002	优先级	P1
	测试步骤	1. 登录后检查侧边栏 2. 点击折叠按钮 3. 非主管账号登录	
	预期结果	主管可见"我的管理"菜单; 非主管不显示; 折叠时宽度60px	

TC-UI-003: 主题切换

测试编号	TC-UI-003	优先级	P2
	测试步骤	1. 点击顶栏太阳/月亮图标 2. 刷新页面	
	预期结果	亮色↔暗色即时切换; 刷新后保持选择(从localStorage读取)	

TC-UI-004: 数据看板图表

测试编号	TC-UI-004	优先级	P1
	测试步骤	1. 导航到数据看板 2. 切换图表Tab 3. 调整浏览器窗口	
	预期结果	4个图表正确渲染(折线图/柱状图/双Y轴/进度条); 窗口resize时图表自适应	

TC-UI-005: Excel导出

测试编号	TC-UI-005	优先级	P2
	测试步骤	1. 进入明细查询→人员 2. 点击导出Excel	
	预期结果	浏览器自动下载Excel文件, 文件名包含导出类型	

3.7 小程序端测试用例

TC-MP-001: 登录与Token持久化

测试编号	TC-MP-001	优先级	P0
	测试步骤	1. 小程序输入用户名密码登录 2. 关闭小程序再打开	
	预期结果	重新打开后自动恢复token, 无需重新登录	

TC-MP-002: 入库列表与详情

测试编号	TC-MP-002	优先级	P1
	测试步骤	1. 首页→入库管理 2. 切换Tab(全部/待入库/已完成) 3. 点击某条记录查看详情	

测试编号	TC-MP-002	优先级	P1
预期结果	创建时间倒序; 详情弹窗显示入库明细和操作按钮		

TC-MP-003: 创建入库

测试编号	TC-MP-003	优先级	P1
测试步骤	1. 入库管理→点"+" 2. 填写入库类型/批次/执行人 3. 提交		
预期结果	创建成功, 跳回列表, 新记录出现		

TC-MP-004: 执行入库(小程序)

测试编号	TC-MP-004	优先级	P1
测试步骤	1. 入库详情→点"确认执行入库" 2. 从库位列表勾选 3. 确认		
预期结果	库位按储存条件过滤, 确认后入库完成, 列表状态更新		

TC-MP-005: 质检完成

测试编号	TC-MP-005	优先级	P1
测试步骤	1. 质检管理→待检列表 2. 点开质检详情 3. 质检员点"完成质检" 4. 选结果+填原因 5. 提交		
预期结果	质检完成, 列表状态变为"已完成"		

TC-MP-006: 数据看板图表

测试编号	TC-MP-006	优先级	P2
测试步骤	1. 首页→数据看板 2. 查看4个Canvas图表		
预期结果	折线图/柱状图/双轴图/进度条正确渲染		

TC-MP-007: 任务中心

测试编号	TC-MP-007	优先级	P1
测试步骤	1. 底部Tab→任务 2. 查看6种任务类型的处理		
预期结果	入库跳转到入库页; 出库/质检/盘点/调整/次品内联完成		

3.8 安全测试用例

TC-SEC-001: 未登录访问受保护页面

测试编号	TC-SEC-001	优先级	P0
	测试步骤	1. Web端清除token后访问 /#/home 2. 小程序清除storage后访问首页	
	预期结果	自动跳转到登录页	

TC-SEC-002: 非主管访问管理页

测试编号	TC-SEC-002	优先级	P1
	测试步骤	1. 用仓库员工账号登录 2. 直接URL访问 /#/manage	
	预期结果	路由守卫拦截, 跳回首页; 侧边栏不显示"我的管理"	

TC-SEC-003: 密码传输加密验证

测试编号	TC-SEC-003	优先级	P0
	测试步骤	1. 抓包查看登录请求 2. 检查password参数	
	预期结果	请求中password为Base64密文, 无法直接读出明文密码	

TC-SEC-004: 库位并发锁定

测试编号	TC-SEC-004	优先级	P1
	测试步骤	1. 创建出库(锁定库位) 2. 同时尝试对同一库位创建另一个出库	
	预期结果	第二个出库无法选择已锁定的库位(lock_status=锁定)	

4. 测试数据准备

4.1 基础测试数据

数据类	内容
测试用户	admin/123456(主管), staff01/123456(仓库员工), qc01/123456(质检员)
测试仓库	一般仓库A(16区128位), 冷冻库B(8区64位), 恒温库C(4区32位)
测试货物	冷冻鸡排(冷冻), 冷藏蔬菜(恒温), 面粉(常温)
测试批次	B20260711001(待入库, 初始100), B20260711002(正常, 剩余50)

4.2 测试环境初始化

1. 执行 `sql/重置数据库.sql` 清空数据
2. 执行 `sql/create_all_tables.sql` 重建表结构
3. 注册测试用户并设置对应position
4. 创建测试仓库(自动生成库区库位)
5. 创建测试货物

5. 测试执行计划

阶段	测试内容	预估时间	负责人
第一阶段	后端接口测试(142个API)	2天	开发自测
第二阶段	核心业务流程功能测试	2天	开发自测
第三阶段	Web前端UI测试	1天	开发自测
第四阶段	微信小程序功能测试	1天	开发自测
第五阶段	安全测试 + 跨平台验证	1天	开发自测
第六阶段	回归测试 + Bug修复验证	1天	开发自测

6. 缺陷管理

6.1 缺陷严重级别

级别	定义	示例
S1 致命	系统崩溃/数据丢失/安全漏洞	入库事务失败导致数据不一致
S2 严重	核心功能不可用	无法创建入库单
S3 一般	功能部分异常, 有替代方案	列表排序不正确
S4 轻微	UI展示问题, 不影响功能	文字对齐偏移

6.2 已知限制

1. 后端权限校验仅在前端实现, Controller层缺少后端角色检查(待加强)
2. 登录接口使用GET方法传参(GET /auth/login?username=&password=), 密码出现在URL中不够安全(建议改用POST)
3. 小程序端 `api.createUser` 函数未在 `utils/api.js` 中导出, 注册功能可能报错
4. 测试覆盖不完整: 仅有一个Spring上下文加载测试, 缺少Service层和Controller层单元测试

(附: 测试用例执行跟踪表见项目测试报告)

